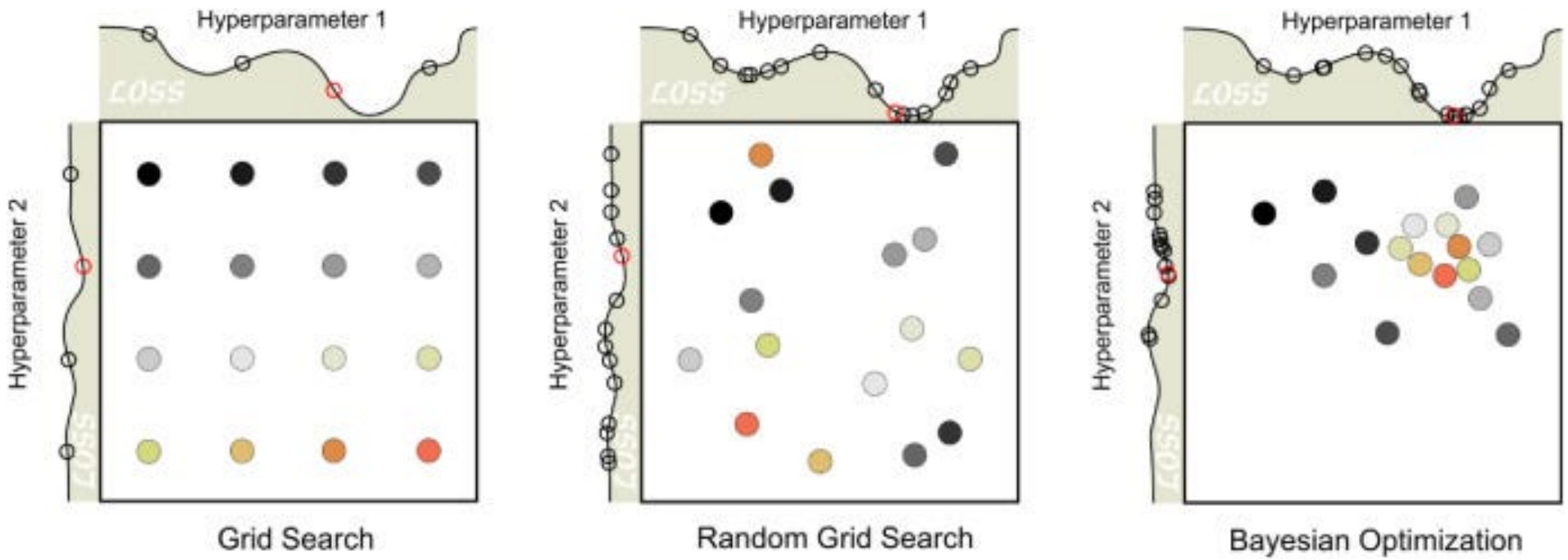
AutoML, Autoresearch, MLOps +@

26.4.8

서민교

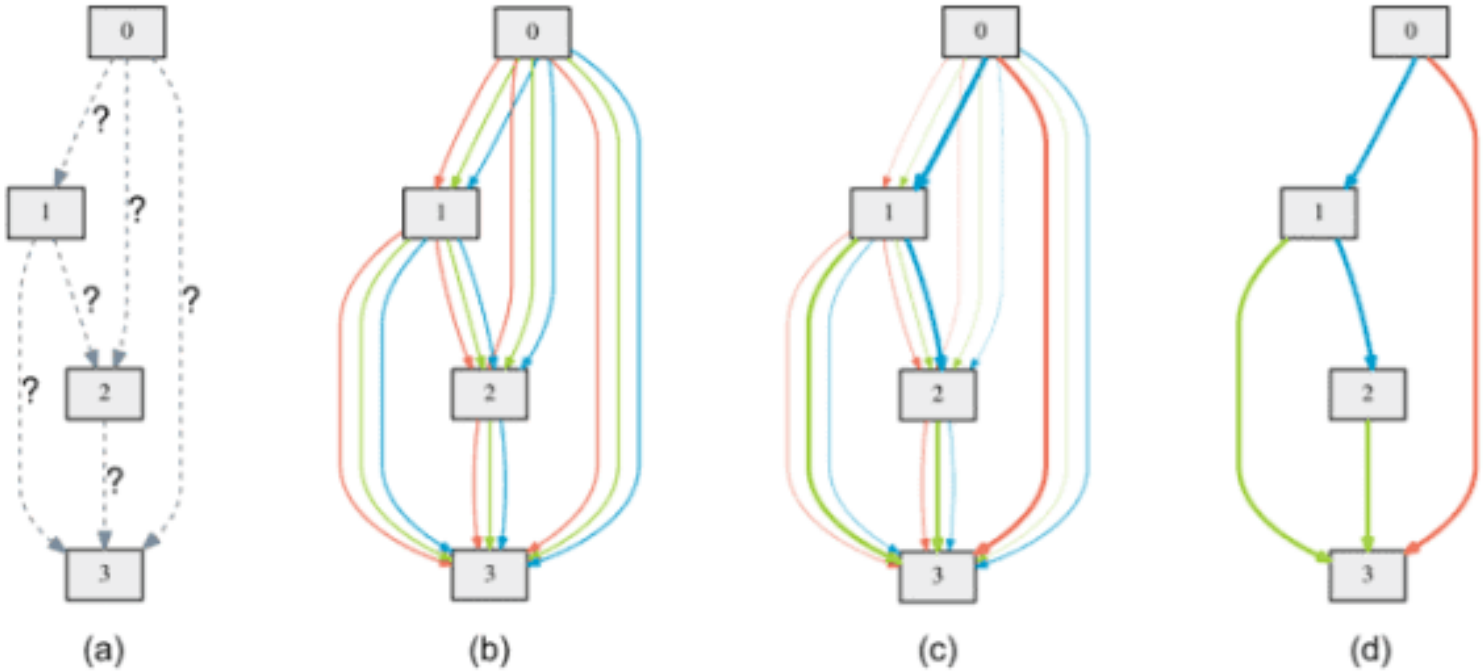
1. AutoML이란 무엇인가?

- 모델 개발 탐색 일부 자동화
- 대표 대상: `model selection`, `hyperparameter tuning`, `pipeline search`
- 핵심: 비교적 주어진 `search space` 안 최적 설정 탐색



2. `Neural Architecture Search`는 AutoML의 확장이다

- parameter 대신 architecture 탐색
- AutoML의 `더 넓은 search space` 확장선
- 그래도 중심은 여전히 모델/파이프라인 후보 탐색



`hyperparameter search → pipeline search → architecture search`

3. `Autoresearch`는 어떻게 등장했고 무엇이 다른가?

- karpathy/autoresearch: 작은 training setup 위 `read → edit → run → keep-or-revert` loop 제시
- 연구 workflow 일부를 agent가 직접 수행하며, 설정 탐색을 넘어 `code`, `module`, `experiment` 자체 수정
- AutoML의 `fixed search space` 바깥으로 확장
- 이후 RD-Agent, AI-Scientist, GPT Researcher 등으로 빠르게 확장

4. Agent 작업 흐름

- 코드 읽기, baseline 파악
- 작은 가설 하나 선택
- 학습 코드나 설정 수정
- 짧은 실험 실행, metric 확인
- 나쁘면 revert, 의미 있으면 keep
- 핵심: `edit 한 번`이 아니라 `짧은 실험 loop의 누적`

```
Question → Read → Edit → Run → Analyze → Next experiment
```

5. AutoML vs. Autoresearch

항목	AutoML	Autoresearch
탐색 대상	config, pipeline, architecture	hypothesis, code, module, experiment
핵심 질문	어떤 설정이 가장 좋은가	다음에 어떤 실험을 해야 하는가
edit 단위	parameter / architecture	code / module / pipeline / experiment
평가 방식	objective 중심	objective + reasoning + iteration
위험	비효율적 탐색	incoherent search, metric hacking
필요한 인프라	experiment infra	experiment + memory + harness

6. 사용례와 확장

사용례

- 문헌 조사 / deep research: GPT Researcher
- 코드 수정 + 실험 반복: karpathy/autoresearch, RD-Agent
- end-to-end 연구 자동화: AI-Scientist

확장

- benchmark / evaluation: MLE-bench, MLAgentBench, MLR-Bench
- plugin / skill 생태계: awesome-autoresearch, Awesome Auto Research Tools
- memory, reusable modules, hardware fork

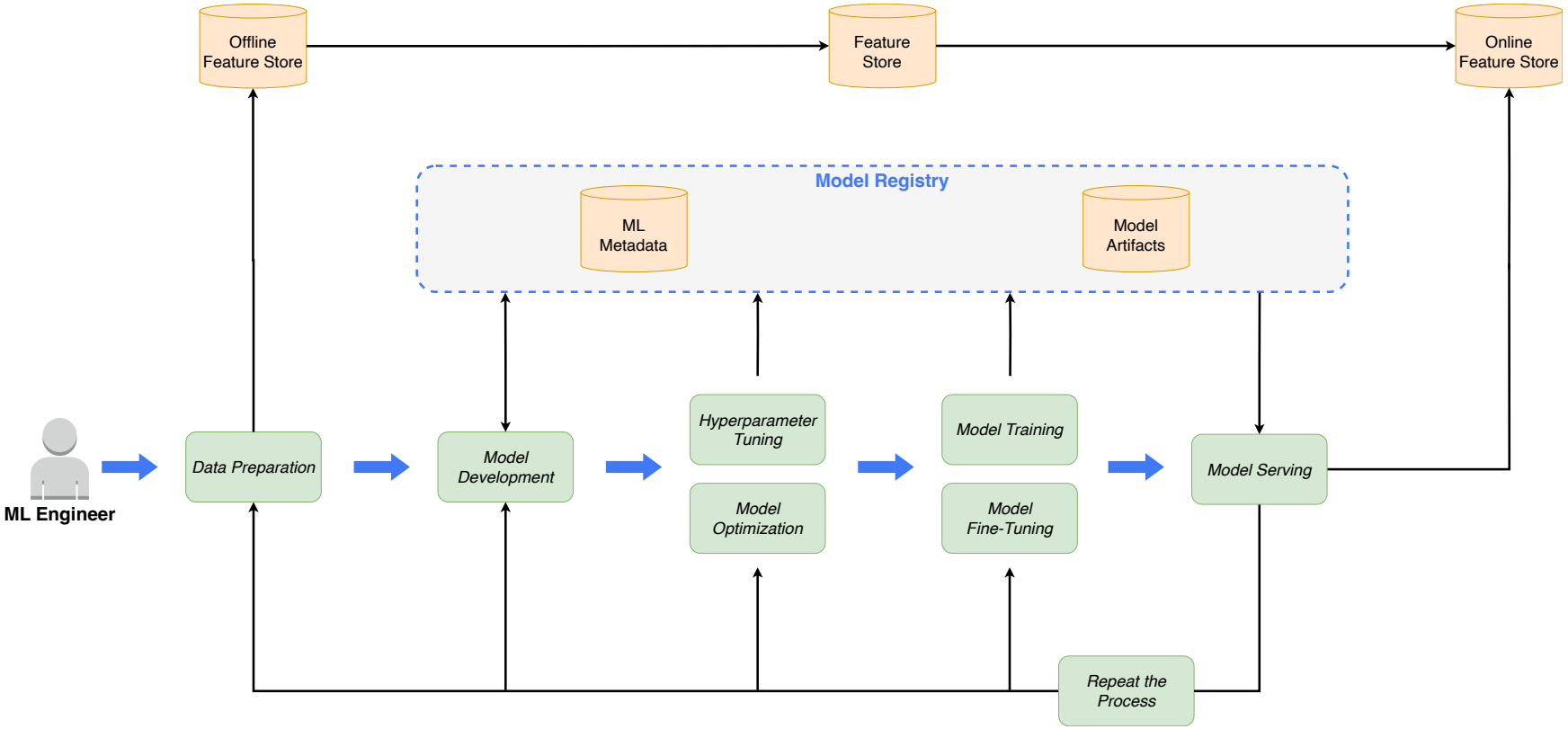
7. 체계적인 실험 관리의 필요

- 공통 문제: `많은 run` 비교와 누적
- 필수 요소: `tracking`, `lineage`, `orchestration`
- agent edit가 들어오면 `artifact`, `promotion`, `monitoring`, `cost control` 중요도 상승
- 결국 운영 문제

8. AutoML과 Autoresearch가 공통으로 요구하는 MLOps 요소

요소	AutoML에서의 역할	Autoresearch에서의 역할
tracking	sweep 비교	hypothesis / code edit history 비교
orchestration	search job 실행	agent + eval job 실행
registry / lineage	best model 승격	experiment / prompt / code provenance 보존
monitoring / cost	retrain trigger, SLO	budget, drift, unsafe promotion guardrail

9. MLOps는 모델 개발, 관리, 배포 파이프라인을 유지 관리하는 작업이다



- Autoresearch loop는 이 큰 ML lifecycle 안의 일부
- 실제 시스템: `data`, `experiment`, `model registry`, `deployment`, `monitoring`
- 핵심 역할: `지속 운영`, `추적`, `승격`, `유지관리`

10. Autoresearch의 단점

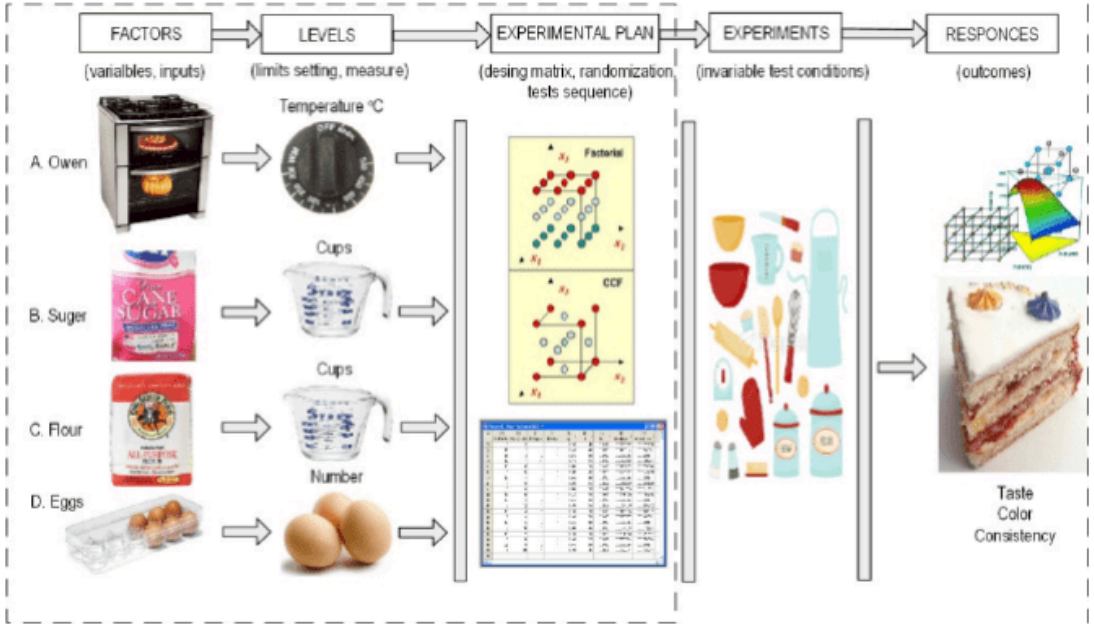
- 실험이 즉흥적으로 이어지기 쉬움
- 왜 이 실험을 했는지 attribution이 약함
- 큰 수정, 작은 튜닝, 검증 실험이 섞이기 쉬움
- robustness, replication, interaction 확인이 뒤로 밀림
- 잘 정리된 random search로 퇴화할 위험

11. 어떤 Harness가 필요한가

- 무엇을 먼저 볼지 정하는 우선순위
- 어떤 조합을 함께 볼지 정하는 규율
- 탐색 단계와 검증 단계 분리
- 작은 수정과 큰 수정을 다르게 다루는 운영 규칙
- 실패도 정보로 남기는 구조
- 다음 라운드를 설계하는 순차 실험 체계

12. Design of Experiments(DoE)란 무엇인가

- 여러 요인을 한 번에 바꿔 보며 effect를 읽는 실험 설계
- 한 번의 최고점보다 `요인`, `상호작용`, `안정성` 파악에 강점
- 핵심 질문: 무엇을 바꿨고, 무엇이 실제로 영향을 줬는가



13. DoE에서 빌려오는 개념

- `screening` : 중요한 요인부터 좁히기
- `factorial thinking` : interaction 보기
- `sequential design` : 라운드별 정교화
- `robust design` : 평균이 아니라 안정성까지 확인
- `mixture / allocation` : 예산과 비율 배분

14. DoE-guided 운영과 비교한 Agents

Agent	운영 방식	특징
`01 Ratchet`	local ratchet loop	incumbent를 branch head로 두고 좁게 mutation
`02 Screening DoE`	simple screening	round마다 한 design question만 분리해 main effect를 읽음
`03 Advanced DoE`	staged DoE program	screening → interaction check → local refinement

15. 실험 설정

항목	설정
benchmarks	`cifar10_real`, `twenty_newsgroups_real`
data budget	CIFAR-10 `max_samples=4000`, 20 Newsgroups `max_samples=8000`
model	`mlp`
agents	`01 Ratchet`, `02 Screening DoE`, `03 Advanced DoE`
execution	dataset × agent별 isolated root `6` 개에서 validation `100` runs 후 hidden finalize

실행 조건

- agent별 isolated root를 따로 만들어 context leakage 없이 독립 실행
- `50-run` block마다 새 subagent를 붙여 context를 reset
- `program.md` 기준 전략을 사용
- validation-only `100` runs를 먼저 누적하고 hidden test는 마지막 `finalize-agent`에서만 공개

열려 있는 주요 축

- preprocessing: `normalization`, `outlier`, `projection`, `resampling`
- architecture: `hidden_dims`, `activation`, `normalization_layer`
- optimization: `solver`, `learning_rate`, `batch_size`, `max_iter`

16. CIFAR-10 결과: validation 탐색과 hidden test

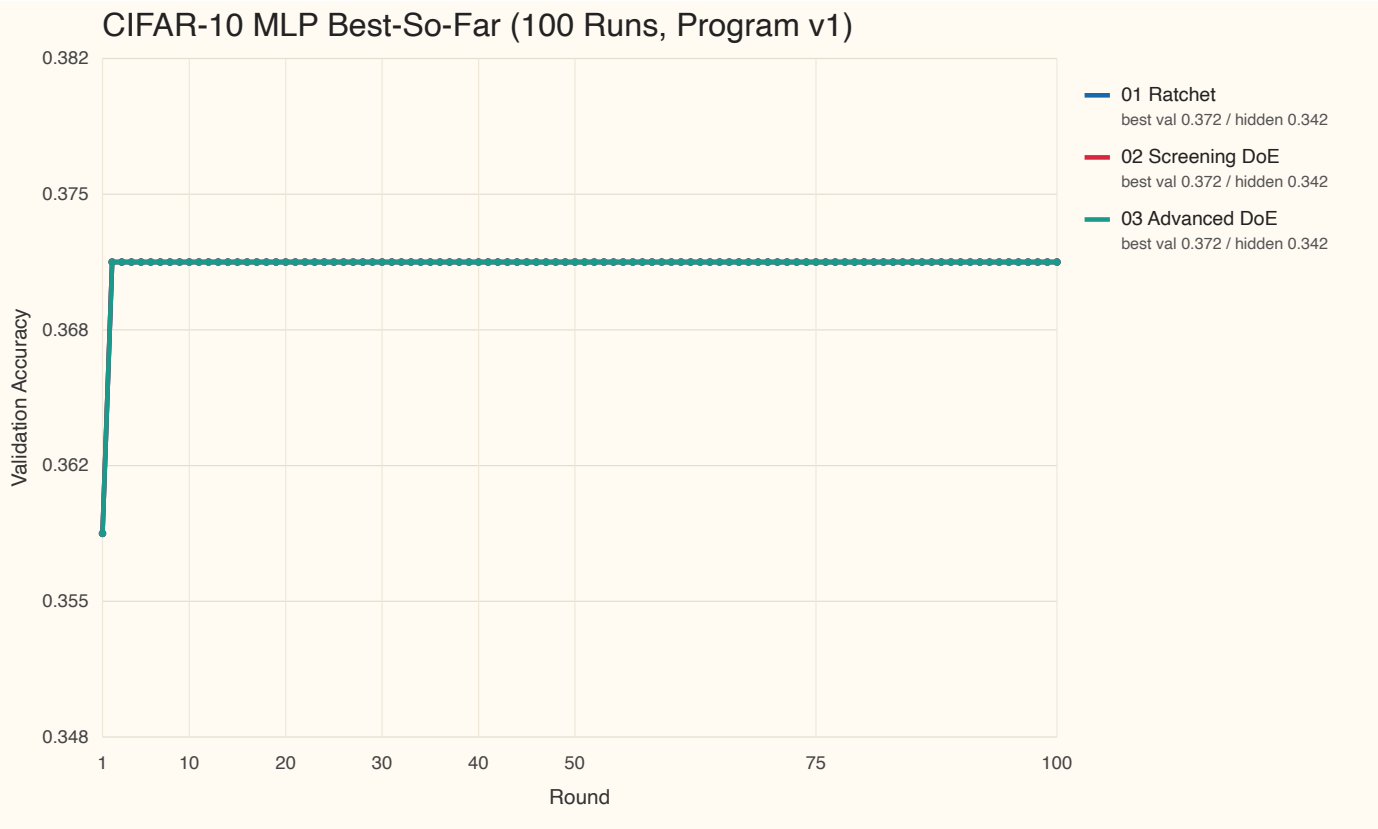
조건: `cifar10_real` / `mlp` / `program.md` batch / validation `100` runs + hidden finalize

Agent	Best Val	Hidden Test	Gap	Run of Best	Incumbent Updates
`01 Ratchet`	`0.3717`	`0.3417`	`0.0300`	`2`	`2`
`02 Screening DoE`	`0.3717`	`0.3417`	`0.0300`	`2`	`2`
`03 Advanced DoE`	`0.3717`	`0.3417`	`0.0300`	`2`	`2`

대표 config

- 세 agent 모두 `standard + [64,32] + relu + adam + wd=5e-4 + lr=1e-3 + bs=64`
- `run 2` 이후 더 좋은 incumbent가 나오지 않았다

17. CIFAR-10 결과: 탐색 궤적



- 세 궤적이 사실상 겹친다.
- 초반 `2` rounds 안에서 같은 basin에 수렴했고, 이후 plateau가 길었다.
- 이번 batch의 차이는 탐색 전략보다 실행 loop와 default prior의 영향이 더 커 보인다.

18. CIFAR-10 해석

- 이번 batch만 보면 전략 차이가 거의 드러나지 않았다.
- 세 agent 모두 같은 config와 같은 score로 수렴했다.
- 따라서 CIFAR에서는 `program.md` 차이보다 executor behavior나 starting prior가 더 지배적이었다.

19. 20 Newsgroups 결과: validation 탐색과 hidden test

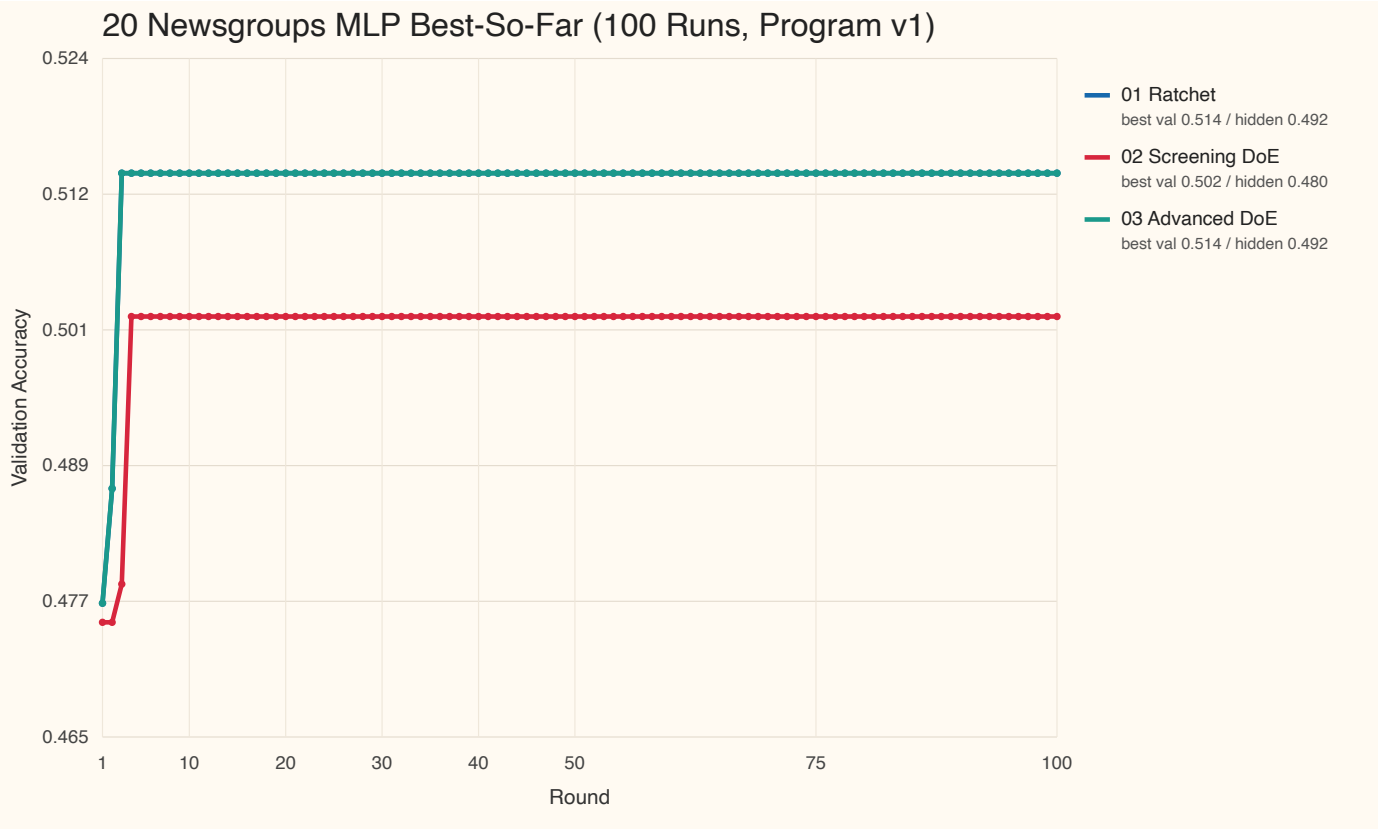
조건: `twenty_newsgroups_real` / `mlp` / `program.md` batch / validation `100` runs + hidden finalize

Agent	Best Val	Hidden Test	Gap	Run of Best	Incumbent Updates
`01 Ratchet`	`0.5142`	`0.4925`	`0.0217`	`3`	`3`
`02 Screening DoE`	`0.5017`	`0.4800`	`0.0217`	`4`	`3`
`03 Advanced DoE`	`0.5142`	`0.4925`	`0.0217`	`3`	`3`

대표 config

- `01 Ratchet`, `03 Advanced DoE`: `standard + [64,32] + tanh + adam + wd=5e-4 + lr=1e-3 + bs=64`
- `02 Screening DoE`: `standard + [64,32] + tanh + lbfgs + wd=5e-4 + lr=1e-3 + bs=64`

20. 20 Newsgroups 결과: 탐색 궤적



- text도 대부분 초반 `3~4` rounds 안에서 best를 찾았다.
- `Screening DoE` 만 `lbfgs` branch로 갈라졌지만 점수는 더 낮았다.
- 결과적으로 text에서도 장기 탐색의 흔적은 약했다.

21. 20 Newsgroups 해석

- text에선 `Ratchet`과 `Advanced DoE`가 같은 basin으로 수렴했다.
- `Screening DoE`만 solver를 바꿨지만 hidden에서도 이득이 없었다.
- 이번 batch는 agent 계약 차이보다 shared default trajectory가 더 강했다.

22. 히스토리에서 남는 지식

`01 Ratchet`

- CIFAR와 text 둘 다 매우 빠르게 baseline 근처 basin으로 수렴했다.
- 이번 batch에서 남는 지식은 “좋은 basin”보다 “어떤 전략이 거의 차이를 못 만들었는가” 쪽이다.

`02 Screening DoE`

- text에서만 `lbfgs` 분기가 나왔지만 hidden 기준 우세는 만들지 못했다.
- screening question 자체가 남았다기보다, shared default를 넘지 못했다는 정보가 남는다.

`03 Advanced DoE`

- staged notes를 갖고도 결과는 ratchet과 거의 같았다.
- strategy richness가 실제 후보 다양성으로 translate되지 않을 수 있다는 사례다.

23. 히스토리 진단

Agent	CIFAR trace	Text trace	읽을 점
`Ratchet`	best `run 2`, updates `2`	best `run 3`, updates `3`	빠른 수렴 후 plateau
`Screening DoE`	best `run 2`, updates `2`	best `run 4`, updates `3`	text에서만 solver branch 분화
`Advanced DoE`	best `run 2`, updates `2`	best `run 3`, updates `3`	풍부한 notes 대비 실제 경로는 거의 동일

- 이번 batch의 핵심 관찰은 `전략 차이의 부재`다.
- `100` rounds를 열어도 실제 유효 탐색은 초반 몇 round에 몰렸다.
- 따라서 문제는 예산 부족보다 executor나 prompt interpretation의 수렴 편향일 가능성이 크다.

24. 요약

- CIFAR: 세 agent 모두 `val 0.3717 / test 0.3417`
- Text: `Ratchet`, `Advanced DoE`는 `val 0.5142 / test 0.4925`, `Screening DoE`는 더 낮았다
- 이번 batch는 “어떤 전략이 이겼는가”보다 “왜 세 전략이 거의 같은 곳으로 갔는가”를 묻는 결과다.

25. 한계

- 이번 결과는 single split 기준이라 분산 추정이 약하다.
- hidden test도 agent당 한 번만 열었으므로 replication이나 confidence interval은 없다.
- fixed subset 위 실험이라 dataset 전체 분포를 대표한다고 보기는 어렵다.
- text는 여전히 fixed TF-IDF representation 위 실험이라 representation search까지 포함한 결론은 아니다.
- 이번 batch에선 세 agent가 거의 같은 최종 config로 수렴해 strategy contrast가 약했다.
- 따라서 `program.md` 차이를 평가하기 전에 executor가 실제로 서로 다른 후보를 충분히 생성하는지부터 다시 검증해야 한다.

26. References

구분	예시
curated landscape	awesome-autoresearch, Awesome Auto Research Tools
end-to-end systems	karpathy/autoresearch, RD-Agent, AI-Scientist
deep research	GPT Researcher
evaluation	MLE-bench, MLAgentBench, MLR-Bench
visuals	AutoML image, NAS image, Kubeflow model registry lifecycle image